

GitHub Actions DSL Compiler

CS 4713 - Compiler Construction | Dr. Ziad Najem | Spring 2026

Abstract

This report showcases a source-to-source compiler for a custom Domain-Specific Language (DSL) targeting GitHub Actions. The compiler translates .gha (custom file extension) programmes into valid .yaml workflow files, enforcing correctness through lexical, syntactic, and semantic analysis before any output is generated. The final system demonstrates full end-to-end functionality: clean code generation for valid inputs and precise diagnostics for both syntax and semantic violations.

1. Team Members

- Fabian C Figueroa Jr. (sop310) - fabian.Figueroa@my.utsa.edu
 - All aspects of this project were designed and implemented independently, including the grammar design, AST construction, semantic analysis, YAML emission, error handling, and validation across all sample inputs.

2. Objective

This project implements a source-to-source compiler for a custom DSL that targets GitHub Actions workflows. Rather than authoring workflow files directly in YAML--which is verbose and can be error-prone at scale--users write programmes in a simplified .gha syntax that prioritises readability. The compiler translates valid input into compliant .yaml output, preserving workflow semantics including trigger events, inter-job dependency declarations, and ordered command execution within each job.

There are two objectives. First, to improve the readability and authoring experience for CI/CD workflows by providing a cleaner input language. Second, to enforce programme correctness through syntax validation and semantic analysis before any output is produced. The compiler therefore functions both as a translator and as a quality gate: only well-formed and semantically valid programmes reach the code generation stage, and invalid programmes receive targeted and actionable diagnostics instead.

From a compiler construction perspective, this project serves as a full pipeline exercise. Rather than stopping at parsing or AST construction, the implementation required integrating each phase into a coherent system where design decisions in the grammar structure, AST modelling, and semantic checking are consistent enough to support reliable output.

3. Approach

The compiler is implemented in Java and uses ANTLR4 to generate the lexer and parser from a grammar specification file (gitActions.g4). ANTLR4 was chosen as it generates efficient LL(*) parsers with built-in error recovery hooks and a clean Visitor API for tree traversal--both of which are critical for building reliable diagnostic abilities. Java was chosen for its strong typing, which made modelling the AST and enforcing boundaries straightforward.

The system follows a classical compiler front-end and back-end pipeline. Source input in .gha format is first tokenised by the ANTLR4-generated lexer, then it is parsed into a concrete parse tree. A custom CompilerErrorListener intercepts any syntax errors and records them with line and column precision. If any syntax errors are present then the compiler terminates immediately--the parse tree is not visited and no AST is constructed.

When parsing succeeds, ASTBuilderVisitor walks the parse tree using the Visitor pattern and produces a typed AST composed of WorkflowNode, JobNode, and StepNode objects. Each node carries a SourceLocation instance encoding its line and column in the original source, which all downstream diagnostic messages will use for precise reports. *Figure 1* below shows the parse tree generated by the ANTLR4-generated parser for a minimal one-job workflow, illustrating how the

grammar rules decompose the source into workflowDeclaration, onClause, and jobDeclaration subtrees before AST construction begins.

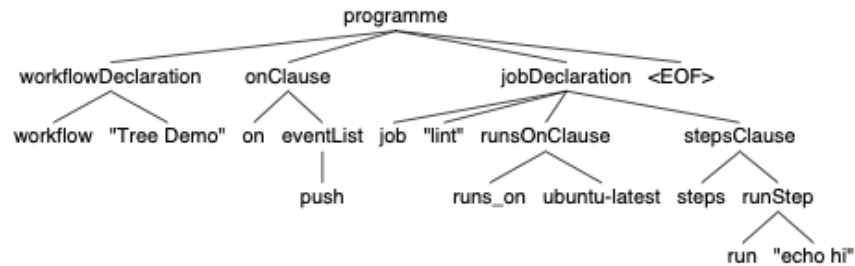


Figure 1: Parse tree produced by the ANTLR4 parser for a minimal one-job .gha workflow. The programme root decomposes into workflowDeclaration, onClause, and jobDeclaration subtrees, each expanding into typed terminal nodes that ASTBuilderVisitor traverses to construct the typed AST.

Semantic analysis runs as a dedicated pass over the completed AST. The SemanticAnalyser checks for five categories of issues: duplicate job names, undefined dependency references in needs clauses, invalid trigger event names, circular dependency chains detected via DFS (depth-first search), and dangerous command patterns in run steps (i.e. rm -rf). Errors block any code generation; warnings are reported but do *not* prevent output.

When the programme passes semantic validation, YAMLEmitter walks the AST and produces a deterministic GitHub Actions .yml file. One explicit normalisation rule is applied at emission time: the DSL keyword runs_on (underscore) is translated to the required YAML key runs-on (hyphen). Staged validation throughout the pipeline ensures that each phase only runs if all prior phases succeeded, preventing cascaded failures and keeping diagnostics clean and interpretable.

Design rationale:

The architecture separates concerns into single-responsibility passes. Parsing is responsible only for structural correctness; AST construction is responsible for internal representation; semantic analysis is responsible for language correctness rules; and emission is responsible for the external target formatting. This separation made debugging significantly easier: when a diagnostic was incorrect, the investigation could be scoped to one phase without needing to reason about the others. It also made iterative changes safer--adjusting a semantic check required no grammar rewrites, and formatting changes in YAMLEmitter had no effect on the parser's behaviour.

4. Key Components

This project is organised around seven components with clearly separated responsibilities, summarised below:

Component	Responsibility
gitActions.g4	Defines all lexer and parser rules; ANTLR4 generates the lexer and LL(*) parser from this grammar file
ASTBuilderVisitor.java	Visits the parse tree using the ANTLR4 Visitor pattern and constructs typed AST nodes with attached source locations
WorkflowNode/JobNode/StepNode	Strongly typed AST nodes representing the workflow, its jobs, and each individual run step in memory
SourceLocation.java	Carries the line and column metadata; attached to every AST node and used by all downstream diagnostic messages
SemanticAnalyser.java	Performs all five semantic checks over the completed AST; records errors and warnings without modifying the tree

Component	Responsibility
YAMLEmitter.java	Walks the validated AST and emits GitHub Actions-compliant YAML, including key normalisation (runs_on -> runs-on)
GitActionsCompiler.java	Main driver; manages the compilation pipeline and halts execution at the first sign of syntax or semantic failure.

In order of execution, the interaction is strictly sequential: gitActions.g4 drives parse-tree construction; ASTBuilderVisitor transforms the tree into typed node objects; SemanticAnalyser validates the model using SourceLocation metadata from each node; and YAMLEmitter generates output YAML only after GitActionsCompiler confirms that no blocking diagnostics were recorded. This linear dependency chain--where each component consumes the well-defined output of the previous one--was fundamental to making the system testable and safe to iterate on during development.

5. Challenges

Indentation-insensitive grammar:

Although the DSL is intended to feel readable in an indentation-friendly style, implementing strict indentation tokenisation with ANTLR4 would have required a custom INDENT/DEDENT pre-processing stage before the lexer--a significant complexity to be added. The grammar was instead designed to be indentation-insensitive, with block structure conveyed by explicit keyword delimiters (i.e. job, steps, run). Whitespace and newlines are discarded entirely via ANTLR skip rules. This choice kept the grammar simple and unambiguous while remaining usable for the current language scope. For a larger grammar in a future iteration, explicit INDENT/DEDENT tokens would be worth attempting..

Syntax error propagation into AST construction:

ANTLR4's default error recovery allows it to produce a parse tree even after syntax errors, inserting error nodes where the grammar could not match. Early versions of the compiler passed these malformed trees to ASTBuilderVisitor, which caused null pointer exceptions and unpredictable crashes during AST construction. The fix was to track whether any syntax error had been reported inside CompilerErrorListener, and to check that flag in GitActionsCompiler immediately after parsing--terminating the pipeline before any AST traversal if errors were present. This made syntax failure behaviour clean and deterministic. The fix was validated with malformed workflow declarations that had previously caused crashes.

Cycle detection and path reconstruction:

Detecting circular job dependencies required DFS over the job dependency graph. The initial implementation correctly identified back edges that indicate cycles, but reconstructed the cycle path in reverse order by traversing a parent map from the entry node. The resulting diagnostic listed the jobs in the wrong order, making it rather difficult to interpret. The fix was to apply Collections.reverse() after path reconstruction and to add a null-safety guard on the parent map lookup to prevent an infinite loop when the cycle entry node had no recorded parent. Cycle detection was validated with several intentionally cyclic job graphs before and after the fix.

Semantic checks: five categories of validation:

Beyond cycle detection, four additional semantic checks were implemented. Duplicate job name detection uses a name-to-node map that is built during a first pass over the AST; the second occurrence of any name triggers an error referencing both source locations. Undefined needs references are checked after all job names are collected, so forward references are permitted and only genuinely undefined names are flagged. Invalid trigger event names are validated against a fixed set of known GitHub Actions event identifiers; unrecognised events produce a warning rather than an error, since the target platform may evolve in the future. Dangerous command detection scans run step strings for patterns like rm -rf and similarly destructive shell commands, emitting a warning to flag potentially harmful workflows without blocking the compilation outright.

Keyword and identifier conflicts:

Values like `ubuntu-latest` contain hyphens, which are not normally valid in identifiers. Because the ANTLR4 IDENT rule initially defined identifiers as `[a-zA-Z_][a-zA-Z0-9_]*`, the tokeniser could not match these runner labels correctly. The rule was extended to allow hyphens in only non-initial positions: `[a-zA-Z_][a-zA-Z0-9_-]*`. This resolved the conflict without affecting keyword matching, as ANTLR4 gives keyword rules priority over the general IDENT rule.

runs_on vs. runs-on:

The DSL uses `runs_on` (underscore) to avoid conflict with ANTLR4's identifier character rules, while GitHub Actions YAML requires `runs-on` (hyphen) in the generated output. This mismatch was not visible during parsing or semantic analysis--both forms tokenise correctly and no semantic rule references the key name directly. The issue only surfaced during output validation, when generated YAML files were compared against expected outputs and tested against GitHub's schema. The fix is a single explicit translation in `YAMLEmitter`, applied whenever a `runs_on` attribute is written to the output file.

6. Sample Input/Output

Validation was performed with three samples that each target a different stage of the pipeline. Testing was performed incrementally--each stage was exercised in isolation first, then as part of full end-to-end runs--to ensure that fixes addressed specific failure modes rather than accidentally masking other issues.

- *python-pipeline-minimal.gha* - a valid three-job workflow (`lint --> test --> deploy`) with a linear dependency chain. Demonstrates successful end-to-end translation from DSL to YAML, including correct needs declarations, `runs-on` normalization, and ordered step emission. Expected result: compilation SUCCESS with YAML emitted.
- *semantic-invalid.gha* - a syntactically valid file that intentionally triggers all five semantic checks: an unrecognised trigger event (`invalid_event`), a duplicate job name (`test`), two undefined needs references (`nonexistent` and `undefined_job`), a circular dependency (`circular_a <--> circular_b`), and a dangerous shell command (`rm -rf /var/www/*`). The compiler reports all findings with source locations and withholds YAML output. Expected result: compilation FAILED, no YAML emitted.
- *syntax-invalid.gha* - the workflow name is written without quotes (`workflow Python Pipeline` instead of `workflow "Python Pipeline"`), causing the parser to fail. The custom error listener reports the exact line and column of the offending token and the pipeline terminates before AST construction. Expected result: parse error at line 3, clean termination.

The DSL-to-YAML translation for the valid sample is illustrated below. Note the `runs_on` --> `runs-on` normalisation and the structure of the emitted needs arrays:

.gha input	.yaml output
<pre> workflow "Python Pipeline" on push, pull_request job "lint" runs_on ubuntu-latest steps run "flake8 src/" job "test" runs_on ubuntu-latest needs lint steps run "pytest tests/" job "deploy" runs_on ubuntu-latest needs test steps run "bash deploy.sh" </pre>	<pre> name: Python Pipeline on: [push, pull_request] jobs: lint: runs-on: ubuntu-latest steps: - run: flake8 src/ test: runs-on: ubuntu-latest needs: [lint] steps: - run: pytest tests/ deploy: runs-on: ubuntu-latest needs: [test] steps: - run: bash deploy.sh </pre>

Outcomes across all three samples:

Sample	Expected Outcome	Result
python-pipeline-minimal.gha	Valid compilation; YAML emitted	PASS
semantic-invalid.gha	All findings reported; no YAML emitted	PASS
syntax-invalid.gha	Syntax error at line 3; clean exit	PASS

The observed behaviour matched the expected behaviour across all three samples. The valid case confirms end-to-end generation quality; the two invalid cases confirm that the compiler fails in controlled and informative ways. Robust failure behaviour--where invalid inputs produce useful diagnostics rather than crashes or silent bad output--was treated as equally important to successful compilation throughout the project.